

```
setting.py
1 import os
2 import openai
3 import secret
4 openai.api_key=secret.api_key
5
6 # WRITE YOUR CODE HERE
7 response = openai.Completion.create(
8     model="text-davinci-002",
9     prompt="",
10    temperature=0,
11    max_tokens=25,
12    top_p=0,
13    frequency_penalty=0,
14    presence_penalty=0
15 )
16
17 prompts ="when did dadism start?"
18
19 #response = openai.Completn.create(
20 # model="text-davinci-002",
21 # prompt=prompts,
22 # temperature=0.1)
23
24 response = openai.Completion.create(
25     model="text-davinci-002",
26     prompt=prompts,
27     temperature=1)
28
29 print(response['choices'][0]['text'].strip())
```

Guide

Collapse

Check Your Settings

arguments control how deterministic the model is when generating a response. That is, adjusting the values of these keyword arguments can create a result that does not change much (if at all) each time the model runs.

Let's set the temperature to `0.1` and try it with the prompt `when did dadism start` . This value increases the model's confidence in its top choice. The closer your temperature is to 0, the more deterministic the model will become. This means you may see very little variance or "creativity" in the response. Instead, you should see a rather matter-of-fact statement.

```
prompts ="when did dadism start?"

response = openai.Completion.create(
    model="text-davinci-002",
    prompt=prompts,
    temperature=0.1)

print(response['choices'][0]['text'].strip())
```

TRY IT!

Dadism started in the early 20th century.

Try this variation:

- Try running the code multiple times when you set `temperature` to `1` . You should now see more variance or "creativity" in the responses.

```
response = openai.Completion.create(
    model="text-davinci-002",
    prompt=prompts,
    temperature=1)
```

TRY IT!

The term "Dadism" was coined in the early 20th century

Now let's take a look at `top_p` . This keyword argument sets the scope of the potential results. The larger the value, the greater number of potential responses the model will consider the best result. Set the value of `top_p` to `1` and run the code a few times. You should see a variety in responses each time the code runs.

```
response = openai.Completion.create(
    model="text-davinci-002",
    prompt=prompts,
    top_p=1)
```

TRY IT!

Try this variation:

- Try running the code multiple times when you set `top_p` to `0.1` . In this example, you are limiting the responses to the 10% of the possible answers. This means the potential responses are limited in nature and therefore more deterministic.

```
response = openai.Completion.create(
    model="text-davinci-002",
    prompt=prompts,
    top_p=0.1)
```

TRY IT!

Combining `top_p` and `temperature`

Both the `top_p` and `temperature` are correlated, which means using one affects the results of the other. Be careful in how you use these two keyword arguments. Take a look at the example below. The `temperature` keyword argument should maximize creativity. However, `top_p` limits the results to only the top 10%. That means the `top_p` value counteracts the `temperature` value, and the results will be more deterministic rather than creative.

```
response = openai.Completion.create(
    model="text-davinci-002",
    prompt=prompts,
    temperature=1,
    top_p=0.1)
```

TRY IT!

Keep in mind how these two keyword arguments work together. Starting out, it might be best to use either `top_p` or `temperature` to control variance in responses. This way you will not "undo" one keyword argument with the other.

`top_p` is a hyperparameter that controls the cumulative probability cutoff for the set of next possible words the model can choose during generation. If `top_p` is set to 0.95, for example, the model will consider the smallest set of next possible words whose combined probability exceeds 0.95.

Troubleshooting Tips

If you're having trouble getting the API to perform as expected, follow this checklist:

- Is it clear what the intended generation should be?
- Are there enough examples?
- Did you check your examples for mistakes? (The API won't tell you directly)
- Are you using `temperature` and `top_p` correctly?
- Are your other settings being used correctly?

The last tip may seem a bit vague. To illustrate this point, change the prompt so that it is asking for the sentiment of each sentence.

```
prompts ="what is the overall sentiment for each of the following sentences:['i am
```